

Лабораторная работа № 2

Первоначальная настройка git

Курс: Архитектура компьютеров и операционные системы

Студент: Пириев Керем

Группа: НПИбд-02-25

Цель работы

Изучить идеологию и применение средств контроля версий, освоить умения по работе с git.

Выполнение работы

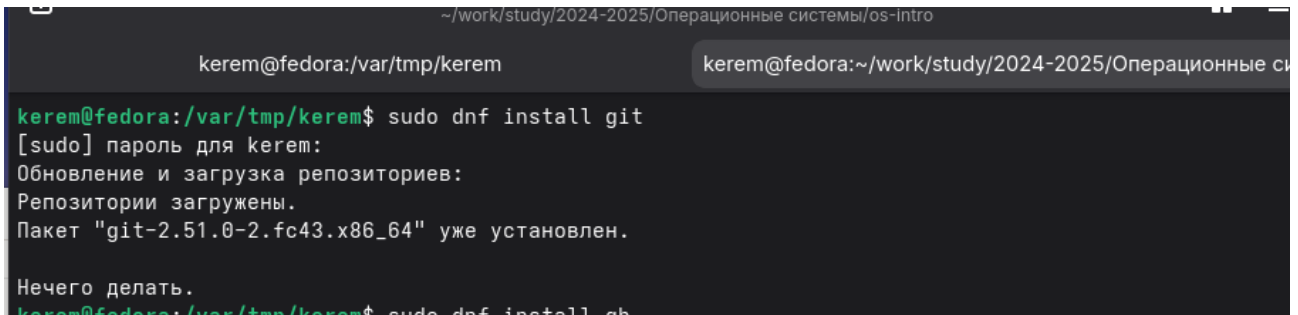
1. Установка программного обеспечения

1.1 Установка Git

Git был установлен командой:

bash

```
sudo dnf install git
```

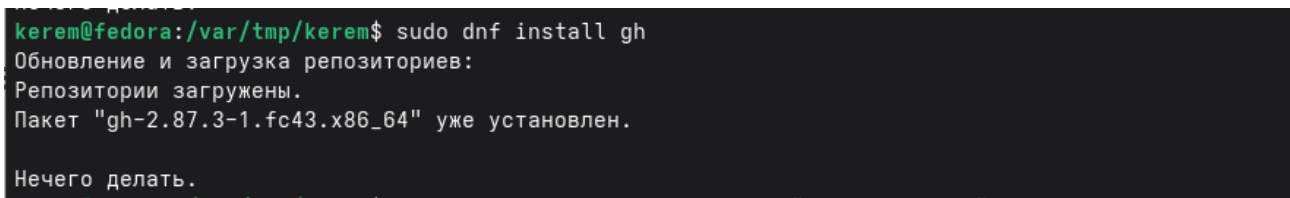


```
kerem@fedora:~/work/study/2024-2025/Операционные системы/os-intro
kerem@fedora:/var/tmp/kerem
kerem@fedora:/var/tmp/kerem$ sudo dnf install git
[sudo] пароль для kerem:
Обновление и загрузка репозитория:
Репозитории загружены.
Пакет "git-2.51.0-2.fc43.x86_64" уже установлен.
Нечего делать.
kerem@fedora:/var/tmp/kerem$ sudo dnf install gh
```

1.2 Установка GitHub CLI (gh)

bash

```
sudo dnf install gh
```



```
kerem@fedora:/var/tmp/kerem$ sudo dnf install gh
Обновление и загрузка репозитория:
Репозитории загружены.
Пакет "gh-2.87.3-1.fc43.x86_64" уже установлен.
Нечего делать.
```

2. Базовая настройка git

Были выполнены команды:

bash

```
git config --global user.name "Kerem Piriev"
git config --global user.email "piriyewkerem144@gmail.com"
git config --global core.quotepath false
git config --global init.defaultBranch master
git config --global core.autocrlf input
git config --global core.safecrlf warn
```

```
kerem@fedora:/var/tmp/kerem$ git config --global user.name "kerem piriyeu"
kerem@fedora:/var/tmp/kerem$ git config --global user.email "piriyewkerem144@gmail.com"
kerem@fedora:/var/tmp/kerem$ git config --global core.quotepath false
kerem@fedora:/var/tmp/kerem$ git config --global init.defaultBranch master
kerem@fedora:/var/tmp/kerem$ git config --global core.autocrlf input
kerem@fedora:/var/tmp/kerem$ git config --global core.safecrlf warn
```

3. Создание SSH ключей

3.1 Создание ключа ed25519

bash

```
ssh-keygen -t ed25519 -C "piriyewkerem144@gmail.com"
```

```
kerem@fedora:/var/tmp/kerem$ ssh-keygen -t rsa -b 4096
Generating public/private rsa key pair.
Enter file in which to save the key (/home/kerem/.ssh/id_rsa):
/home/kerem/.ssh/id_rsa already exists.
Overwrite (y/n)? y
Enter passphrase for "/home/kerem/.ssh/id_rsa" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/kerem/.ssh/id_rsa
Your public key has been saved in /home/kerem/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:qRibr2gj6BYub7ebyEre/rWVBQ2GnBmHfzi2NdFjAuk kerem@fedora
The key's randomart image is:
+---[RSA 4096]-----+
|      ..*=0 .      |
|      *0.00 +      |
|      0...+ .      |
|      E.+          |
|      .  S =..      |
|      = . .0        |
|0.. + .. 0         |
|*+=0.0. 0          |
|=0*=*=..           |
+---[SHA256]-----+
```

3.2 Просмотр публичного ключа

bash

cat ~/.ssh/id_ed25519.pub

```
kerem@fedora:/var/tmp/kerem$ ssh-keygen -t ed25519
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/kerem/.ssh/id_ed25519):
/home/kerem/.ssh/id_ed25519 already exists.
Overwrite (y/n)? y
Enter passphrase for "/home/kerem/.ssh/id_ed25519" (empty for no passphrase)
Enter same passphrase again:
Your identification has been saved in /home/kerem/.ssh/id_ed25519
Your public key has been saved in /home/kerem/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:dts673IlnG+xvpbo6swNwa3H5ewklmc9NPY8H+1ptDA kerem@fedora
The key's randomart image is:
+--[ED25519 256]--+
|
|
|
|      . .
|     S +....+
|    . . *+E=++
|    + B=XB*
|   oo*o***+
|   .*0=+=oo
+-----[SHA256]-----+
```

Вывод:

text

```
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIL3tr6LSFxUIhYnDdYkxZE9CXcgdVJvPMJs6DpLzRcVo
piriyewkerem144@gmail.com
```

4. Создание PGP ключа (GPG)

bash

gpg --full-generate-key

```
kerem@fedora:/var/tmp/kerem$ gpg --full-generate-key
gpg (GnuPG) 2.4.8; Copyright (C) 2025 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
```

Параметры ключа:

- Тип: RSA and RSA
- Размер: 4096
- Срок действия: 0 (бессрочно)
- Имя: Kerem Piriev
- Email: piriyewkerem144@gmail.com

```

Выберите тип ключа:
(1) RSA and RSA
(2) DSA and Elgamal
(3) DSA (sign only)
(4) RSA (sign only)
(9) ECC (sign and encrypt) *default*
(10) ECC (только для подписи)
(14) Existing key from card
Ваш выбор? 1
длина ключей RSA может быть от 1024 до 4096.
Какой размер ключа Вам необходим? (3072) 4096
Запрошенный размер ключа - 4096 бит
Выберите срок действия ключа.
    0 = не ограничен
    <n> = срок действия ключа - n дней
    <n>w = срок действия ключа - n недель
    <n>m = срок действия ключа - n месяцев
    <n>y = срок действия ключа - n лет
Срок действия ключа? (0)
Срок действия ключа не ограничен
Все верно? (y/N) y

GnuPG должен составить идентификатор пользователя для идентификации ключа.

Ваше полное имя: kerem
Адрес электронной почты: piriyewkerem144@gmail.com
Примечание:
Вы выбрали следующий идентификатор пользователя:
    "kerem <piriyewkerem144@gmail.com>"

Сменить (N)Имя, (C)Примечание, (E)Адрес; (O)Принять/(Q)Выход? o
Необходимо получить много случайных чисел. Желательно, чтобы Вы
в процессе генерации выполняли какие-то другие действия (печать
на клавиатуре, движения мыши, обращения к дискам); это даст генератору
случайных чисел больше возможностей получить достаточное количество энтропии.

```

4.1 Просмотр созданных ключей

bash

```
gpg --list-secret-keys --keyid-format LONG
```

Вывод:

text

```
sec  rsa4096/XXXXXXXXXX 2026-05-26 [SC]
```

uid [ultimate] Kerem Piriev <piriyewkerem144@gmail.com>

Скриншот: [вставьте скриншот списка ключей]

```
kerem@fedora:/var/tmp/kerem$ gpg --list-secret-keys --keyid-format LONG
gpg: проверка таблицы доверия
gpg: marginals needed: 3 completes needed: 1 trust model: pgp
gpg: глубина: 0 достоверных: 3 подписанных: 0 доверие: 0-, 0q, 0n, 0m, 0f, 3u
[keyboxd]
-----
sec  rsa4096/366909230651655D 2026-06-04 [SC]
      2CC66FBEA3629F16082EC3F3366909230651655D
uid      [ абсолютно ] kerem <piriyewkerem144@gmail.com>
ssb  rsa4096/C383A6496BD20BCF 2026-06-04 [E]

sec  ed25519/D56AB2BC50B05D5A 2026-05-26 [SC]
      A0FAF8978C263375F5EF7A59D56AB2BC50B05D5A
uid      [ абсолютно ] kerem <1032254162@rudn.ru>
ssb  cv25519/3A8BD0233E8D0B61 2026-05-26 [E]

sec  rsa4096/1EB978C2DFBA7060 2026-06-04 [SC]
      DFF934D7CE196744EBC0E5D81EB978C2DFBA7060
uid      [ абсолютно ] kerem <piriyewkerem144@gmail.com>
ssb  rsa4096/61346D0CAFAAD1B6 2026-06-04 [E]
```

5. Добавление SSH ключа на GitHub

1. SSH ключ скопирован командой:

```
bash
```

```
cat ~/.ssh/id_ed25519.pub
```

2. Ключ добавлен на GitHub: **Settings** → **SSH and GPG keys** → **New SSH key**

```
kerem@fedora:~/work/study/2024-2025/Операционные системы$ cat ~/.ssh/id_ed25519.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIPYmv73ESbLkXevoy2n0cR75p00EkspRBMbHFXSqPxce kerem@fedora
kerem@fedora:~/work/study/2024-2025/Операционные системы$ ssh-keygen -t ed25519 -C "piriyewkerem144@gmail.com"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/kerem/.ssh/id_ed25519):
/home/kerem/.ssh/id_ed25519 already exists.
Overwrite (y/n)? y
Enter passphrase for "/home/kerem/.ssh/id_ed25519" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/kerem/.ssh/id_ed25519
Your public key has been saved in /home/kerem/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:N0vJ8yB1xRp92xPQWlxxSA4TovppnqDt9H5kcmUcbQ piriyewkerem144@gmail.com
The key's randomart image is:
+--[ED25519 256]--+
|      .o .++|
|      +. o =|
|      =.oE + |
|      ooo= o |
|      .S % . .|
|      oo% 0 . o.|
|      ..o* 0 o =|
|      . =. . + o.|
|      .+.o . .|
+-----[SHA256]-----+
```

6. Добавление GPG ключа на GitHub

1. GPG ключ экспортирован:

```
bash
```

```
gpg --armor --export <отпечаток>
```

2. Ключ добавлен на GitHub: **Settings** → **SSH and GPG keys** → **New GPG key**

```
kerem@fedora:~/work/study/2024-2025/Операционные системы$ cat ~/.ssh/id_ed25519.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIL3tr6lSFxUIhYnDdYkxZE9CXcgdVJvPMJsGDplzRcVo piriyeuwerem144@gmail.com
kerem@fedora:~/work/study/2024-2025/Операционные системы$ ssh -T git@github.com
Hi piriyeuwerem! You've successfully authenticated, but GitHub does not provide shell access.
```

7. Настройка подписи коммитов git

```
bash
```

```
git config --global user.signingkey <отпечаток>
git config --global commit.gpgsign true
```

```
git config --global gpg.program $(which gpg2)
```

```
kerem@fedora:~/work/study/2024-2025/Операционные системы/os-intro$ git config --global commit.gpgsign true
kerem@fedora:~/work/study/2024-2025/Операционные системы/os-intro$ git config --global user.signingkey <отпечаток>
bash: синтаксическая ошибка рядом с неожиданным маркером «newline»
kerem@fedora:~/work/study/2024-2025/Операционные системы/os-intro$ git config --global commit.gpgsign true
kerem@fedora:~/work/study/2024-2025/Операционные системы/os-intro$ git config --global gpg.program $(which gpg2)
kerem@fedora:~/work/study/2024-2025/Операционные системы/os-intro$
```

8. Создание репозитория на GitHub

Репозиторий создан: https://github.com/piriyewkerem/study_2025-2026_os-intro

github.com/piriyewkerem/study_2025-2026_os-intro

piriyewkerem / study_2025-2026_os-intro

Code Issues Pull requests Agents Actions Projects Wiki Security and quality Insights Settings

study_2025-2026_os-intro Public

Pin Watch 0 Fork 0 Star 0

main

Go to file + <> Code

kerem144pc Add Makefile 479c0ac · 43 minutes ago

COURSE	Initial commit: add README	50 minutes ago
Makefile	Add Makefile	43 minutes ago
README.md	Initial commit: add README	50 minutes ago

README

About

No description, website, or topics provided.

Activity

0 stars

0 watching

0 forks

Releases

No releases published

[Create a new release](#)

Packages

No packages published

[Publish your first package](#)

Contributors 1

kerem144pc

Suggested workflows

Based on your tech stack

9. Клонирование репозитория

bash

```
mkdir -p ~/work/study/2024-2025/"Операционные системы"
cd ~/work/study/2024-2025/"Операционные системы"
git clone git@github.com:piriyewkerem/study_2025-2026_os-intro.git os-intro
```

```
kerem@fedora:~/work/study/2024-2025/Операционные системы/os-intro$ mkdir -p ~/work/study/2024-2025/"Операционные системы"
kerem@fedora:~/work/study/2024-2025/Операционные системы/os-intro$ cd ~/work/study/2024-2025/"Операционные системы"
kerem@fedora:~/work/study/2024-2025/Операционные системы$ git clone git@github.com:piriyewkerem/study_2025-2026_os-intro.git os-intro
fatal: целевой путь «os-intro» уже существует и не является пустым каталогом.
kerem@fedora:~/work/study/2024-2025/Операционные системы$
```

10. Настройка каталога курса

bash

```
cd os-intro
echo os-intro > COURSE
```

```
git add .
git commit -m "Initial commit"
git push
```

```
kerem@fedora:~/work/study/2024-2025/Операционные системы$ cd os-intro
kerem@fedora:~/work/study/2024-2025/Операционные системы/os-intro$ echo os-intro > COURSE
kerem@fedora:~/work/study/2024-2025/Операционные системы/os-intro$ git add .
kerem@fedora:~/work/study/2024-2025/Операционные системы/os-intro$ git commit -m "Initial commit"
Текущая ветка: main
Эта ветка соответствует «origin/main».

нечего коммитить, нет изменений в рабочем каталоге
kerem@fedora:~/work/study/2024-2025/Операционные системы/os-intro$ git push
Everything up-to-date
kerem@fedora:~/work/study/2024-2025/Операционные системы/os-intro$
```

Выводы

В ходе выполнения лабораторной работы я:

- Установил и настроил Git и GitHub CLI
- Сгенерировал SSH и GPG ключи
- Добавил ключи в аккаунт GitHub
- Настроил автоматическую подпись коммитов
- Создал удалённый репозиторий и клонировал его локально
- Настроил рабочее пространство для выполнения лабораторных работ

Таким образом, цель работы достигнута: я освоил базовые навыки работы с системой контроля версий Git.

Контрольные вопросы

1. Что такое системы контроля версий (VCS) и для решения каких задач они предназначаются?

Системы контроля версий — это программные инструменты, которые отслеживают изменения в файлах, позволяют хранить историю изменений, возвращаться к предыдущим версиям, работать над проектом нескольким разработчикам одновременно, разрешать конфликты.

2. Объясните следующие понятия VCS и их отношения: хранилище, commit, история, рабочая копия.

- **Хранилище (repository)** — база данных, содержащая все версии файлов.
- **Commit** — фиксация изменений, создающая новую версию.
- **История (history)** — цепочка коммитов, показывающая эволюцию проекта.

- **Рабочая копия (working copy)** — текущая версия файлов, с которой работает пользователь.

3. Что представляют собой и чем отличаются централизованные и децентрализованные VCS? Приведите примеры VCS каждого вида.

- **Централизованные** (CVS, Subversion): есть единый центральный сервер, все изменения отправляются на него.
- **Децентрализованные** (Git, Mercurial): каждый разработчик имеет полную копию репозитория, можно работать без подключения к серверу.

4. Опишите действия с VCS при единоличной работе с хранилищем.

Создание локального репозитория (`git init`), добавление файлов (`git add`), фиксация изменений (`git commit`), просмотр истории (`git log`), просмотр различий (`git diff`).

5. Опишите порядок работы с общим хранилищем VCS.

Клонирование удалённого репозитория (`git clone`), получение изменений (`git pull`), создание коммитов, отправка изменений (`git push`), разрешение конфликтов при необходимости.

6. Каковы основные задачи, решаемые инструментальным средством **git**?

- Отслеживание истории изменений
- Работа с ветками
- Слияние изменений
- Совместная разработка
- Откат к предыдущим версиям

7. Назовите и дайте краткую характеристику командам **git**.

- `git init` — создание нового репозитория
- `git add` — добавление файлов в индекс
- `git commit` — фиксация изменений
- `git push` — отправка изменений на удалённый сервер
- `git pull` — получение изменений с сервера
- `git clone` — копирование удалённого репозитория
- `git branch` — работа с ветками
- `git merge` — слияние веток
- `git log` — просмотр истории

8. Приведите примеры использования при работе с локальным и удалённым репозиториями.

Локальный:

```
bash
```

```
git init  
git add .  
git commit -m "message"
```

Удалённый:

```
bash
```

```
git clone git@github.com:user/repo.git  
git pull  
git push
```

9. Что такое и зачем могут быть нужны ветви (branches)?

Ветки — это независимые линии разработки. Нужны для:

- Разработки новых функций без влияния на основной код
- Исправления ошибок
- Экспериментов
- Параллельной работы нескольких разработчиков

10. Как и зачем можно игнорировать некоторые файлы при commit?

Создаётся файл `.gitignore`, в котором перечисляются шаблоны имён файлов/каталогов. Это нужно, чтобы не включать в репозиторий временные файлы, служебные файлы ОС, файлы с паролями, скомпилированные объектные файлы и т.д.